



## LeetCode 242: Valid Anagram

### 1. Problem Title & Link

**Title:** LeetCode 242: Valid Anagram

**Link:** <https://leetcode.com/problems/valid-anagram/>

### 2. Problem Statement (Short Summary)

Given two strings *s* and *t*, determine whether *t* is an anagram of *s*.

Two strings are anagrams if:

- They contain the same characters
- Character frequencies are identical
- Order does not matter

Return:

true → Anagram

false → Not Anagram

### Example

Input:

*s* = "anagram"

*t* = "nagaram"

Output:

true

Because both strings contain:

a → 3

n → 1

g → 1

r → 1

m → 1

### 3. Examples (Input → Output)

#### Example 1

Input:

*s* = "anagram"

*t* = "nagaram"

Output:

true

**Example 2**

Input:

`s = "rat"``t = "car"`

Output:

`false`**Example 3**

Input:

`s = "listen"``t = "silent"`

Output:

`true`**Example 4**

Input:

`s = "hello"``t = "world"`

Output:

`false`**4. Constraints** $1 \leq s.length, t.length \leq 5 * 10^4$ `s` and `t` consist of lowercase English letters

Important:

Same frequency of every character.

**5. Core Concept (Pattern / Topic)****HashMap / Counting**

Secondary Topics:

- Frequency Array
- String Processing



## 6. Thought Process (Step-by-Step Explanation)

### Understanding Anagrams

Example:

listen

silent

Characters:

l i s t e n

s i l e n t

Same characters.

Same frequencies.

Different order.

Therefore:

Anagram

### Brute Force Idea

Sort both strings.

Example:

listen

Sorted:

eilnst

silent

Sorted:

eilnst

Same.

Therefore:

true

### Problem

Sorting costs:

$O(n \log n)$

Can we do better?

### Better Observation



We only care about:

Character Counts

Example:

anagram

Frequency:

a → 3

n → 1

g → 1

r → 1

m → 1

For:

nagaram

Frequency:

n → 1

a → 3

g → 1

r → 1

m → 1

Same counts.

Therefore:

true

### Key Insight

If every character count matches:

Strings are anagrams.

Use:

HashMap

or

26-size Frequency Array

## 7. Visual / Intuition Diagram (ASCII Diagram)

Example:

anagram

Count:

a → 3

n → 1

g → 1

r → 1



```
m → 1
```

Example:

nagaram

Count:

a → 3

n → 1

g → 1

r → 1

m → 1

Match.

Return:

true

Non-Anagram

rat

Count:

r → 1

a → 1

t → 1

car

Count:

c → 1

a → 1

r → 1

Mismatch.

Return:

false

## 8. Pseudocode (Language Independent)

```
if lengths differ
    return false

create count array of size 26

for each char in s
    increment count

for each char in t
    decrement count

if any count != 0
    return false
```



```
return true
```

## 9. Code Implementation

### Python

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:

        if len(s) != len(t):
            return False

        count = [0] * 26

        for ch in s:
            count[ord(ch) - ord('a')] += 1

        for ch in t:
            count[ord(ch) - ord('a')] -= 1

        for value in count:
            if value != 0:
                return False

        return True
```

### Java

```
class Solution {

    public boolean isAnagram(String s, String t) {

        if(s.length() != t.length())
            return false;

        int[] count = new int[26];

        for(char ch : s.toCharArray()) {

            count[ch - 'a']++;
        }

        for(char ch : t.toCharArray()) {

            count[ch - 'a']--;
        }

        for(int value : count) {

            if(value != 0)
```



```
        return false;
    }

    return true;
}
```

## 10. Time & Space Complexity

### Frequency Array Approach

Time Complexity:  $O(n)$

Single traversal.

Space Complexity:  $O(1)$

Array size always:

26

constant.

## 11. Common Mistakes / Edge Cases

### Mistake 1

Not checking lengths first.

Example:

abc

ab

Impossible to be anagrams.

### Mistake 2

Comparing sets.

Wrong:

`set(s) == set(t)`

Example:

aab

abb

Sets same:

`{a,b}`

But frequencies differ.

**Mistake 3**

Ignoring repeated characters.

Need frequency comparison.

**Edge Cases****Single Character**

Input:

"a"

"a"

Output:

true

**Different Length**

Input:

ab

abc

Output:

false

**Same Characters Different Counts**

Input:

aab

abb

Output:

false

**12. Detailed Dry Run**

Input:

s = "anagram"

t = "nagaram"

Initial:

count[26] = all zeros

After processing:





s

a=3

n=1

g=1

r=1

m=1

Array stores positive counts.

Process:

t

Subtract frequencies.

Final array:

all zeros

Return:

true

### 13. Common Use Cases (Real-Life / Interview)

#### Spell Checkers

Detect rearranged words.

#### Text Analysis

Character frequency comparison.

#### Cryptography

Permutation validation.

#### Interview Purpose

Tests:

HashMap

Counting

Frequency Arrays

### 14. Common Traps (Important!)

#### Trap 1

Using nested loops.

Complexity:



$O(n^2)$

Too slow.

### Trap 2

Using set comparison.

Does not consider frequency.

### Trap 3

Sorting immediately.

Works but not optimal.

## 15. Builds To (Related LeetCode Problems)

LC 242 - Valid Anagram



LC 383 - Ransom Note



LC 49 - Group Anagrams



LC 438 - Find All Anagrams



LC 567 - Permutation in String

These are classic frequency-counting problems.

## 16. Alternate Approaches + Comparison

| Approach        | Time          | Space  | Interview Rating |
|-----------------|---------------|--------|------------------|
| Nested Loops    | $O(n^2)$      | $O(1)$ | Poor             |
| Sorting         | $O(n \log n)$ | $O(1)$ | Good             |
| HashMap         | $O(n)$        | $O(n)$ | Good             |
| Frequency Array | $O(n)$        | $O(1)$ | Best             |

### Approach 1: Sorting

`sorted(s) == sorted(t)`

Easy.

But slower.

**Approach 2: HashMap**

Count frequencies.

Works for all characters.

**Approach 3: Frequency Array (Chosen)**

Since only lowercase English letters exist.

Most optimal.

**17. Why This Solution Works (Short Intuition)**

Two strings are anagrams if every character appears the same number of times in both strings.

The frequency array counts occurrences in one string and removes occurrences from the other.

If all counts become zero, the strings are anagrams.

**18. Variations / Follow-Up Questions****Follow-Up 1**

Support Unicode characters.

Use:

HashMap

instead of size-26 array.

**Follow-Up 2**

Group anagrams together.

Related:

LC 49

**Follow-Up 3**

Find all anagrams of a pattern inside a string.

Related:

LC 438

**Follow-Up 4**

Check whether one string is a permutation of another.

Same solution.

**Interview Takeaway**

The key pattern is:

Frequency Counting



Whenever you hear:

Same characters

Character counts

Rearranged strings

Permutation check

think:

HashMap

Frequency Array

This pattern appears frequently in string-processing, sliding-window, and hashing interview problems.